

AI Shortcut Examples — Full Guide

24 practical examples for building features, debugging, backend/frontend development, PostgreSQL/Prisma, documentation, and production review.

How to use: Copy the shortcut block into ChatGPT or Cursor. The `.ai/commands`` files define the behavior behind each shortcut.

1. `🔗` Build a New Feature — Understand First

`/feature /steps`

I want to add a "Customer Follow-up" feature to the Banquet CRM.

A user should be able to:

- create a follow-up
- select an enquiry
- set follow-up date/time
- add notes
- mark follow-up as completed
- see upcoming follow-ups

First understand the existing architecture and identify the files/modules that should change.

Do not implement anything yet.

2. `🔗` Build the Feature — Create the Plan

`/plan /architecture`

Create an implementation plan for this feature.

Include:

- database changes
- backend modules
- API endpoints
- frontend screens/components
- validation
- permissions
- notifications
- tests
- migration

3. `🔗` Implement the Feature

`/implement /testing`

Implement the Customer Follow-up feature according to the approved plan.

Do not modify unrelated modules.

After implementation:

- run/type-check relevant code
- add tests
- list all changed files
- explain how to verify the feature

4. `🔗` Debug a Bug

/debug

Here is the error:

500 Internal Server Error

PrismaClientKnownRequestError
Unique constraint failed

Here is the relevant code:

[paste code]

Find the root cause.

Do not change the code yet.

5. 🐛 Fix the Bug

/fix /testing

Fix the root cause identified above.

Requirements:

- make the smallest safe change
- don't change unrelated behavior
- add or update regression tests
- explain why the bug occurred
- explain why the fix resolves it

6. 📝 Review the Fix

/review /production

Review the fix for:

- correctness
- race conditions
- database behavior
- security
- error handling
- regression risks
- production readiness

7. 📄 Create Feature Documentation

/document /feature

Create documentation for the Customer Follow-up feature.

Document:

- feature purpose
- user workflow
- business rules
- database entities
- API endpoints
- request/response examples
- validation
- permissions
- frontend behavior
- error scenarios
- edge cases
- testing
- deployment considerations

Use the actual implementation and schema from the repository.
Do not invent behavior.

8. 📄 Create a PRD Before Building

/prd

I want to add a Customer Follow-up feature to my Banquet CRM.

Users should be able to:

- schedule follow-ups
- associate them with an enquiry
- add notes
- mark them completed
- see upcoming follow-ups

Create a complete PRD.

Include:

- problem
- objective
- target user
- user journey
- functional requirements
- non-functional requirements
- business rules
- edge cases
- permissions
- analytics
- acceptance criteria
- MVP scope
- future enhancements

9. 📄 Critically Review the PRD

/review /critique

Critically review this PRD.

Look for:

- missing requirements

- ambiguous requirements
- unnecessary complexity
- edge cases
- permission problems
- UX problems
- technical risks

Do not rewrite it yet.

10. 📄 Create a PostgreSQL Schema

/schema /architecture

Design the PostgreSQL schema for Customer Follow-ups.

Requirements:

A follow-up belongs to an enquiry.

A follow-up should contain:

- enquiry
- scheduled date/time
- notes
- status
- completed date/time
- created by
- created at
- updated at

Consider:

- indexes
- foreign keys
- constraints
- enum/status design
- timezone handling
- soft deletion
- audit requirements

Do not write Prisma code yet.
First propose the schema.

11. 📄 Convert Schema to Prisma

/prisma /schema

Convert the approved schema into a Prisma schema.

Check:

- relations
- indexes
- unique constraints
- cascade behavior
- nullable fields
- migration safety

Do not modify unrelated models.

12. 📄 Create a New Backend API

/api /architecture

I need APIs for Customer Follow-ups.

Required operations:

POST /api/follow-ups
GET /api/follow-ups
GET /api/follow-ups/:id
PATCH /api/follow-ups/:id
DELETE /api/follow-ups/:id

Design the API.

Include:

- request schemas
- response schemas
- status codes
- validation
- authentication
- authorization
- pagination
- filtering
- sorting
- error responses

Use the existing backend conventions in the repository.
Do not implement yet.

13. 📁 Implement the Backend API

/implement /backend /api /testing

Implement the Follow-up API using the existing backend architecture.

Requirements:

- Fastify
- TypeScript strict mode
- existing validation approach
- existing authentication
- existing error handling
- Prisma
- existing logging conventions

Do not introduce a new framework or architecture.

Add tests for the endpoints.

14. 📁 Design the Frontend Feature

/ux /ui /feature

Design the frontend experience for Customer Follow-ups.

Users need to:

- see upcoming follow-ups
- filter by date/status
- create a follow-up
- edit it
- mark it completed
- delete it

Consider:

- desktop
- tablet
- mobile
- loading state
- empty state
- error state
- form validation
- confirmation dialogs
- success feedback

Do not write code yet.

15. 🌐 Implement the Frontend

/react /frontend /implement /testing

Implement the Follow-up frontend.

Use the existing:

- React architecture
- routing
- UI components
- API client
- state/data-fetching approach
- form components
- notification system

Do not introduce new libraries unless necessary.

Add tests for important user flows.

16. 📁 Backend + Frontend Full-Stack Feature

/fullstack /feature /testing

Build a complete Customer Follow-up feature.

Backend:

- PostgreSQL
- Prisma
- Fastify
- TypeScript

Frontend:

- React
- TypeScript
- existing application architecture

Requirements:

- create follow-up
- list follow-ups
- update follow-up
- complete follow-up
- delete follow-up

First analyze the repository and create an implementation plan.

DO NOT IMPLEMENT YET.

17. 📁 Backend and Frontend Separately

/backend /api /testing

Implement the Customer Follow-up API.

First inspect the existing API architecture.

Implement:

- POST /follow-ups
- GET /follow-ups
- GET /follow-ups/:id
- PATCH /follow-ups/:id
- DELETE /follow-ups/:id

Use existing conventions.

Do not modify frontend code.

Then, in the frontend project:

/frontend /react /testing

Implement the Customer Follow-up UI using the existing API.

Do not modify backend code.

18. □ Add Authentication / Permission

/security /architecture

Design authorization rules for Customer Follow-ups.

Requirements:

- regular sales users can access their permitted enquiries
- managers can access team enquiries
- unauthorized users must not access follow-up data

First analyze the existing authentication/authorization system.

Do not implement yet.

19. □ Something Broke After a Change

/debug /rootcause

I implemented [feature].

Before the change:
[behavior]

After the change:
[behavior]

Error:
[paste error]

Find what changed and identify the root cause.

Inspect git diff/history if available.

Do not fix yet.

20. □ Refactor Existing Code

/refactor /review /testing

Refactor this module.

Goals:

- reduce complexity
- improve readability
- remove duplication
- improve testability

Constraints:

- preserve behavior
- preserve API contract
- don't change database schema
- don't introduce new dependencies

Before changing code:

- identify current problems
- explain proposed refactoring

Then implement it and update tests.

21. □ Architecture Decision

/architecture /compare /recommend

I am considering two approaches:

A. Add this as a module in the existing modular monolith.

B. Create a separate microservice.

Feature:

Customer Notification Service

Analyze:

- development complexity
- deployment
- database
- scalability
- failure isolation
- observability
- team complexity
- cost
- future migration

Recommend one approach for the current stage of the product.

22. 📄 Production Incident

/debug /production

Production API is returning intermittent 500 errors.

Symptoms:

[paste symptoms]

Logs:

[paste logs]

Recent deployment:

[details]

Analyze:

1. likely root cause
2. impact
3. immediate mitigation
4. permanent fix
5. verification
6. prevention

Do not make speculative changes.

Clearly separate confirmed facts from hypotheses.

23. 📄 Database Migration Review

/prisma /migration /production /review

Review this Prisma migration.

Check:

- data loss
- locking
- downtime
- backwards compatibility
- indexes

- foreign keys
- existing production data
- rollback strategy

Tell me whether it is safe to deploy.

24. ☐ Recommended End-to-End Workflow

```
/understand  
/prd  
/architecture  
/plan  
/schema /prisma  
/backend /api  
/frontend /react  
/testing  
/review /security /performance  
/production  
/document
```

Use this sequence for a significant new full-stack feature.
At each stage, use the actual repository as the source of truth.
Do not skip architecture, validation, tests, or production review.